

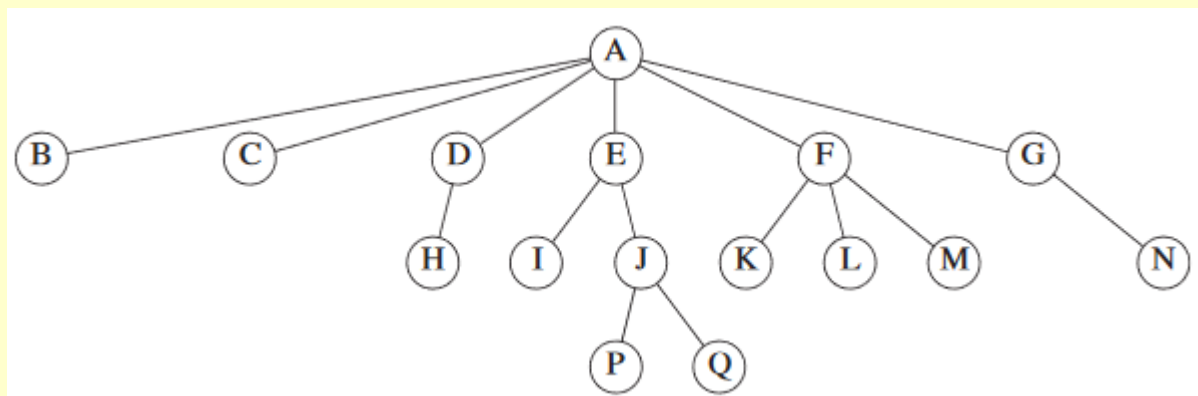
Binary Search Trees

Modified version of
<https://www.cs.unc.edu/~plaisted/com>
[p550/15-btrees.ppt](#)

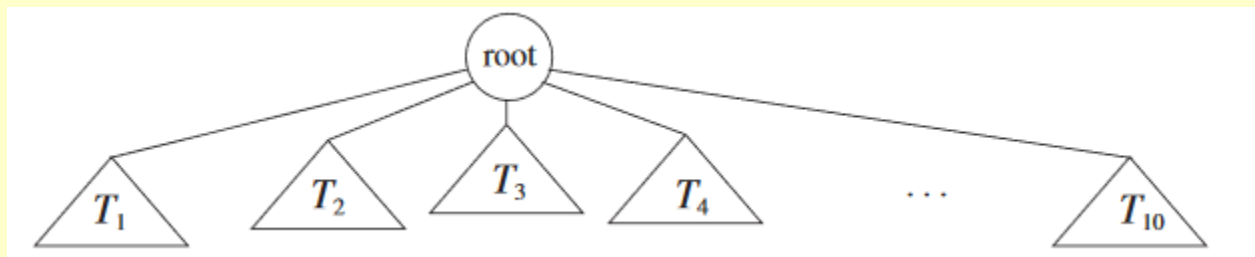
Questions added from GATE

- ◆ *For large amounts of input, the linear access time of linked lists is prohibitive – weiss*

Forms of trees(balanced BST, 2-3 trees) provide an average running time of $O(\log n)$ for most operations.



Root node: A
Child of D: H
Parent of D: A



Each node has
a subtree under it.

Depth: The depth of a node is the number of edges in the unique path **from the root to the node**. Root depth=0

*Depth of a tree is equal to the depth of the deepest leaf.

Height: The height of a node is the length of the longest path **from node to a leaf**. All leaves are at height 0.

*The height of a tree is equal to the height of the root.

Depth of a tree is equal to the height of the tree.

- ♦ **Ancestor-Descendant:** If there is a path from n_1 to n_2 , then n_1 is an ancestor of n_2 and n_2 is a descendant of n_1 . If $n_1 = n_2$, then n_1 is a proper ancestor of n_2 and n_2 is a proper descendant of n_1

Implementation

- ◆ When number of children is not known

Struct TreeNode

```
{ char * string;
```

```
TreeNode *firstChild;
```

```
TreeNode *nextSibling;
```

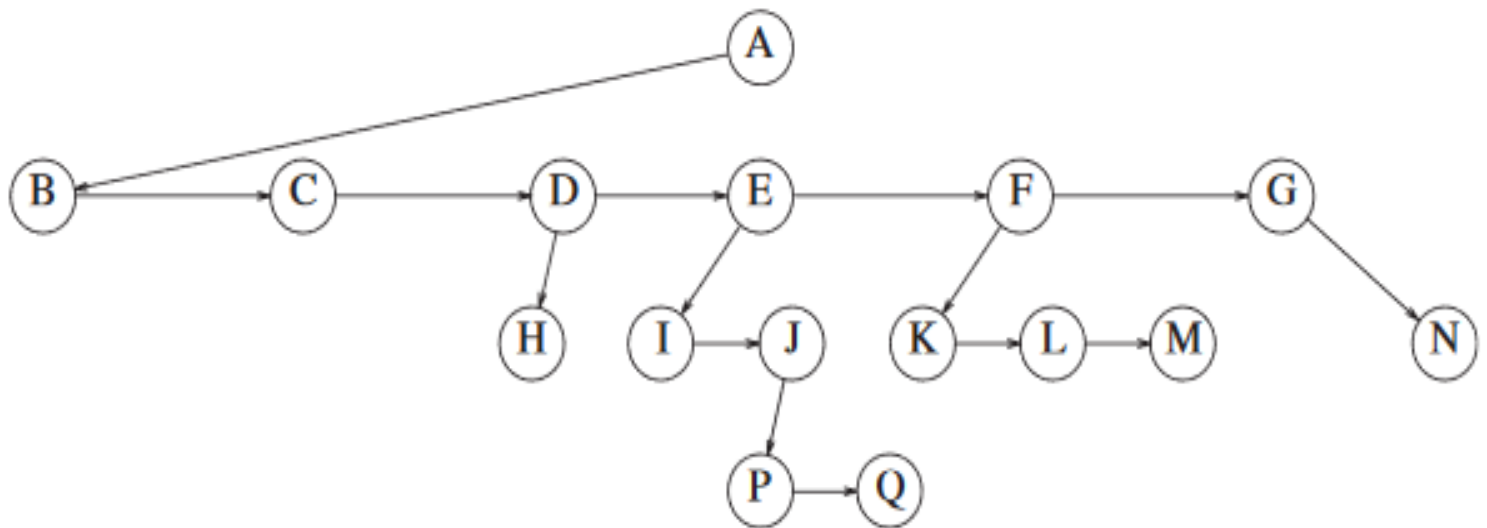
```
}
```

Node:

value: data

left: pointer to left child

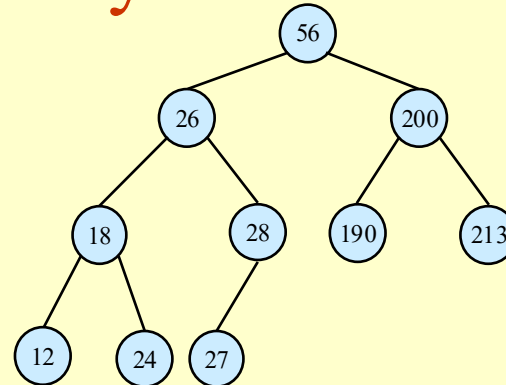
right: pointer to right child



Binary Trees

- ◆ Recursive definition
 1. An empty tree is a binary tree
 2. A node can have not more than two children.

binary tree



Binary Tree Implementation

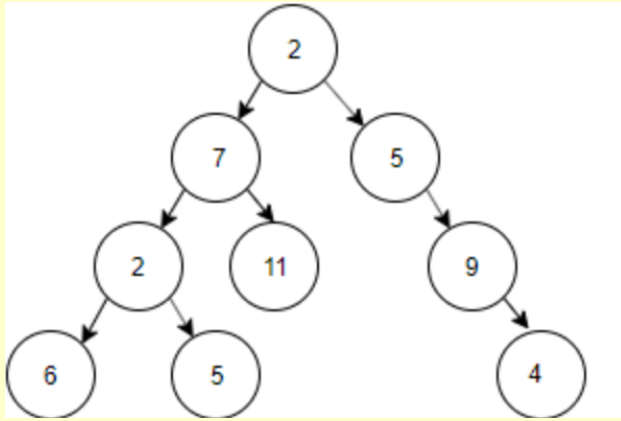
- ◆ When number of children is atmost two.

```
Struct TreeNode
{ char * string;
TreeNode *firstChild;
TreeNode *secondChild;
}
```

- ◆ Max depth of a binary tree of n nodes ?

Traversals

- ◆ Inorder : Left Root Right
- ◆ Preorder : Root Left Right
- ◆ Postorder: Left Right Root



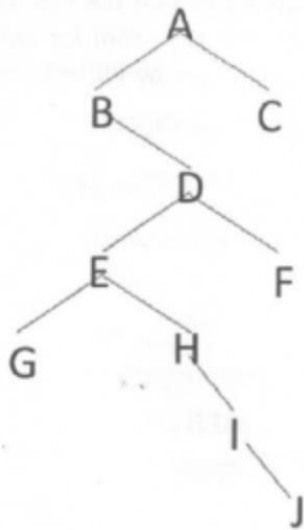
Write out the inorder, post order and pre order traversal of the above figure

Assume the digits 7, 5, 1, 8, 3, 6, 0, 9, 4, 2 are added in that sequence into a binary search tree that is initially empty. The binary search tree follows the standard natural number ordering. What is the resulting tree's inorder traversal sequence?

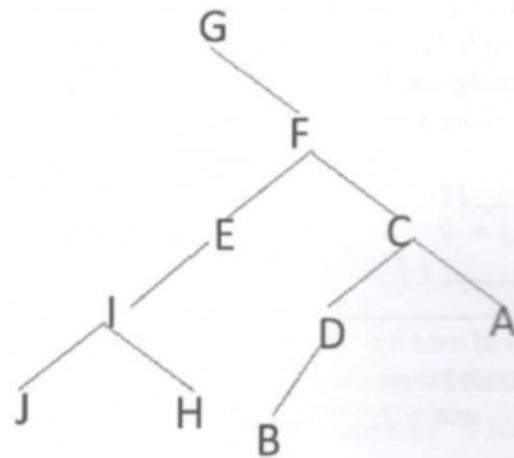
Consider the following data. The pre order traversal of a binary tree is A, B, E, C, D. The in order traversal of the same binary tree is B, E, A, D, C. The level order sequence for the binary tree is _____

If the trees Tree-1 and Tree-2 are the ones listed below:

Tree-1



Tree-2



Which ordering of each tree gives the same sequence

The in-order traversal of a tree resulted in FBGADCE.

Then the pre-order traversal of that tree would result in

FGBDECA

ABFGCDE

BFGCDEA

AFGBDEC

- ◆ Inorder : Left Root Right
- ◆ Preorder : Root Left Right
- ◆ Postorder: Left Right Root

For a binary tree T, let the preorder traversal be [1, 2, 7, 3, 4, 5, 6] and inorder traversal be [2, 7, 1, 4, 3, 6, 5]. Can you construct the tree T.

Gate 2008

The following three are known to be the preorder, inorder and postorder sequences of a binary tree. But it is not known which is which.

- I. *MBCAFHPYK*
- II. *KAMCBYPFH*
- III. *MABCKYFPH*

Pick the true statement from the following.

- A. I and II are preorder and inorder sequences, respectively
- B. I and III are preorder and postorder sequences, respectively
- C. II is the inorder sequence, but nothing more can be said about the other two sequences
- D. II and III are the preorder and inorder sequences, respectively

More definitions

- ♦ A **full binary tree** (sometimes proper **binary tree** or **2-tree**) is a **tree** in which every node other than the leaves has two children.

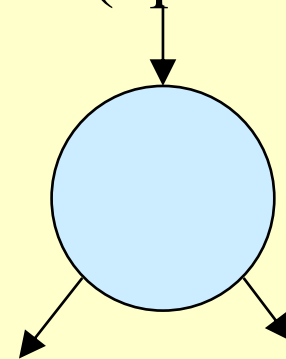
A **complete binary tree** is a **binary tree** in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible.

Binary Search Trees

- ♦ View today as data structures that can support **dynamic set operations**.
 - » Search, Minimum, Maximum, Predecessor, Successor, Insert, and Delete.
- ♦ Can be used to build
 - » **Dictionaries.**
 - » **Priority Queues.**
- ♦ Basic operations take time proportional to the height of the tree – $O(h)$.

BST – Representation

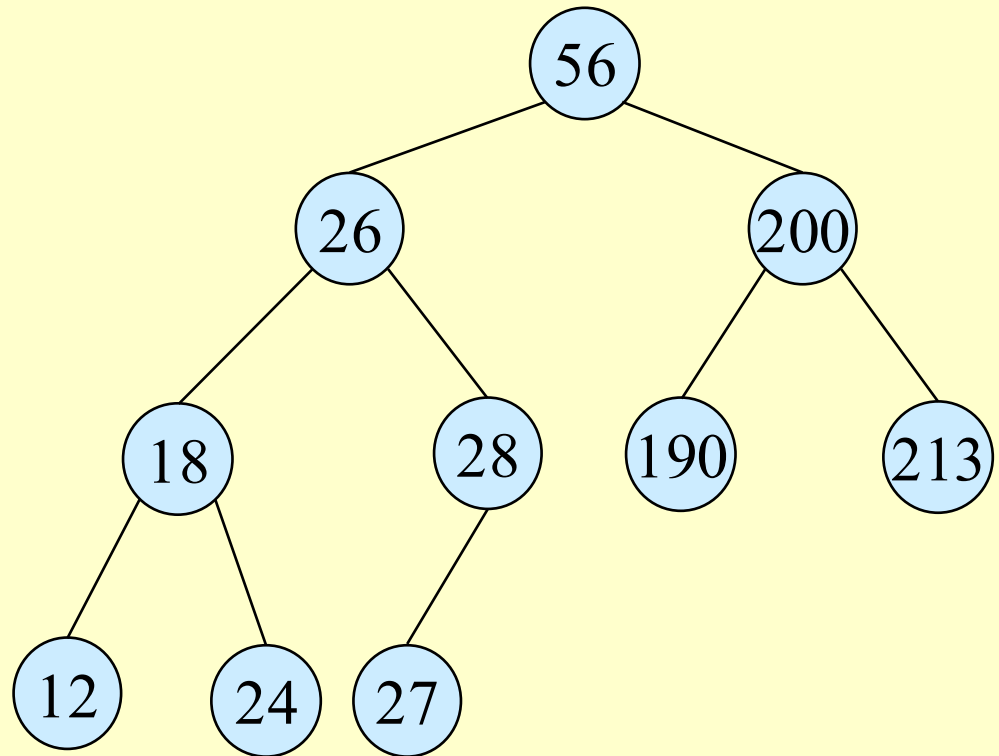
- ◆ Represented by a linked data structure of nodes.
- ◆ *root(T)* points to the root of tree T .
- ◆ Each node contains fields:
 - » *key*
 - » *left* – pointer to left child: root of left subtree.
 - » *right* – pointer to right child : root of right subtree.
 - » *p* – pointer to parent. $p[\text{root}[T]] = \text{NIL}$ (optional).



Binary Search Tree Property

- ◆ Stored keys must satisfy the *binary search tree* property.

- » $\forall y$ in left subtree of x , then $key[y] \leq key[x]$.
- » $\forall y$ in right subtree of x , then $key[y] \geq key[x]$.



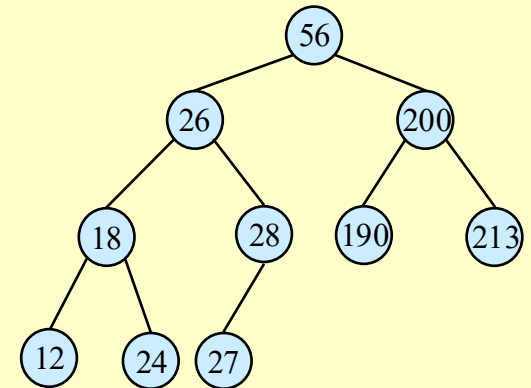
Given a BST, how can I
Print the sorted element set?

Inorder Traversal

The binary-search-tree property allows the keys of a binary search tree to be printed, in (monotonically increasing) order, recursively.

Inorder-Tree-Walk (x)

1. **if** $x \neq \text{NIL}$
2. **then** Inorder-Tree-Walk($\text{left}[x]$)
3. print $\text{key}[x]$
4. Inorder-Tree-Walk($\text{right}[x]$)



♦ How long does the walk take?

Operations

- ◆ Querying for a key
- ◆ Find Min, Max
- ◆ Find Successor, Predecessor
- ◆ Insert, Delete

Querying a Binary Search Tree

Querying a Binary Search Tree

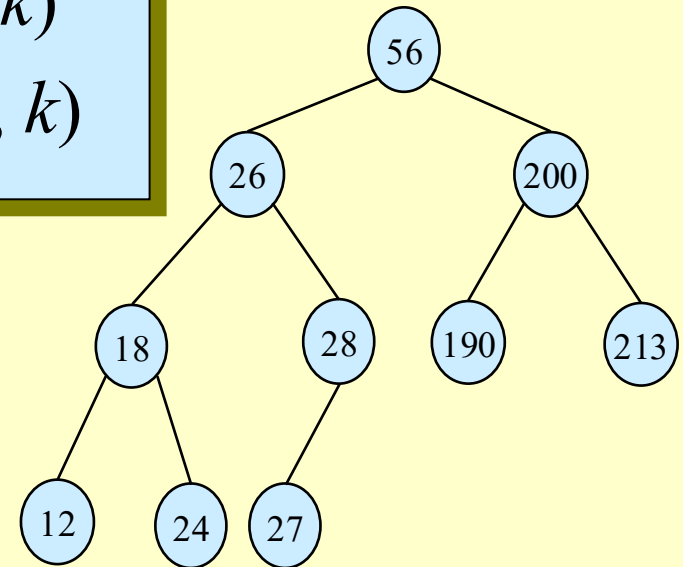
- ♦ All dynamic-set search operations can be supported in $O(h)$ time.
- ♦ $h = \Theta(\lg n)$ for a balanced binary tree (and for an average tree built by adding nodes in random order.)
- ♦ $h = \Theta(n)$ for an unbalanced tree that resembles a linear chain of n nodes in the worst case.

Tree Search

Tree-Search(x, k)

1. **if** $x = \text{NIL}$ *or* $k = \text{key}[x]$
2. **then** return x
3. **if** $k < \text{key}[x]$
4. **then** return Tree-Search($\text{left}[x], k$)
5. **else** return Tree-Search($\text{right}[x], k$)

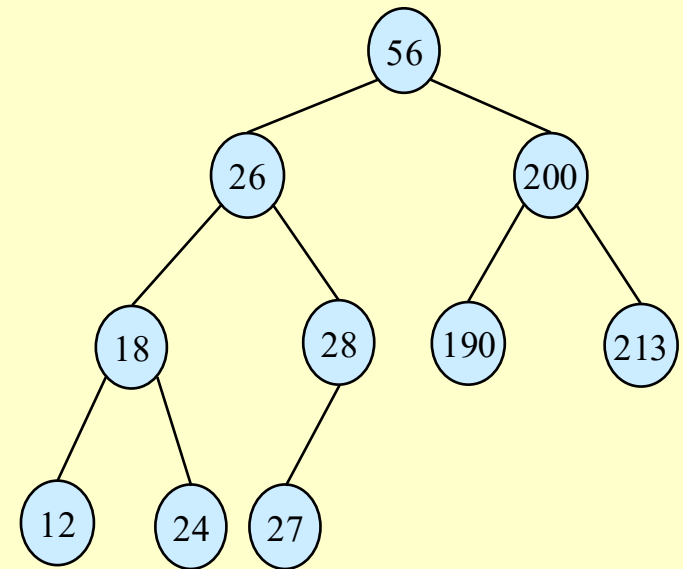
Running time: $O(h)$



Iterative Tree Search

Iterative-Tree-Search(x, k)

1. **while** $x \neq NIL$ **and** $k \neq key[x]$
2. **do if** $k < key[x]$
3. **then** $x \leftarrow left[x]$
4. **else** $x \leftarrow right[x]$
5. **return** x



The iterative tree search is more efficient on most computers.
The recursive tree search is more straightforward.

Finding Min & Max

Finding Min & Max

- ♦ The binary-search-tree property guarantees that:
 - » The **minimum** is located at the ?
 - » The **maximum** is located at the ?
- ♦ Try writing the code

Q: How long do they take?

Finding Min & Max

- ♦ The binary-search-tree property guarantees that:
 - » The **minimum** is located at the **left-most** node.
 - » The **maximum** is located at the **right-most** node.

Tree-Minimum(x)

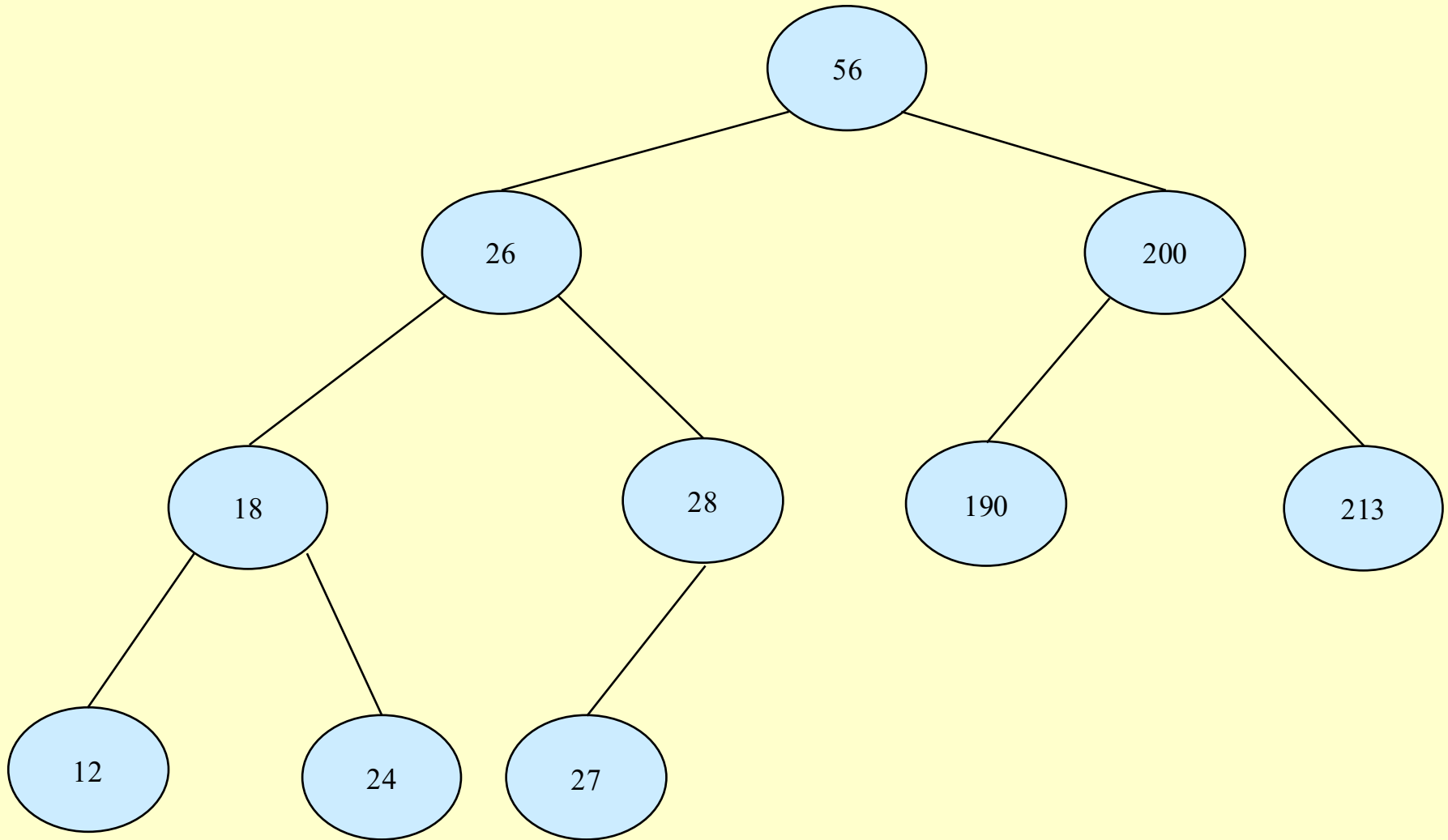
```
1. while  $left[x] \neq NIL$   
2.   do  $x \leftarrow left[x]$   
3. return  $x$ 
```

Tree-Maximum(x)

```
1. while  $right[x] \neq NIL$   
2.   do  $x \leftarrow right[x]$   
3. return  $x$ 
```

Q: How long do they take?

Predecessor and Successor



Predecessor and Successor

- ◆ Successor of node x is the node y such that $key[y]$ is the smallest key greater than $key[x]$.
- ◆ The successor of the largest key is NIL.
- ◆ Search consists of two cases.
 - » If node x has a non-empty right subtree, then x 's successor is the minimum in the right subtree of x .
 - » If node x has an empty right subtree, then:
 - As long as we move to the left up the tree (move up through right children), we are visiting smaller keys.
 - x 's successor y is the node that x is the predecessor of (x is the maximum in y 's left subtree).
 - In other words, x 's successor y , is the lowest ancestor of x whose left child is also an ancestor of x .

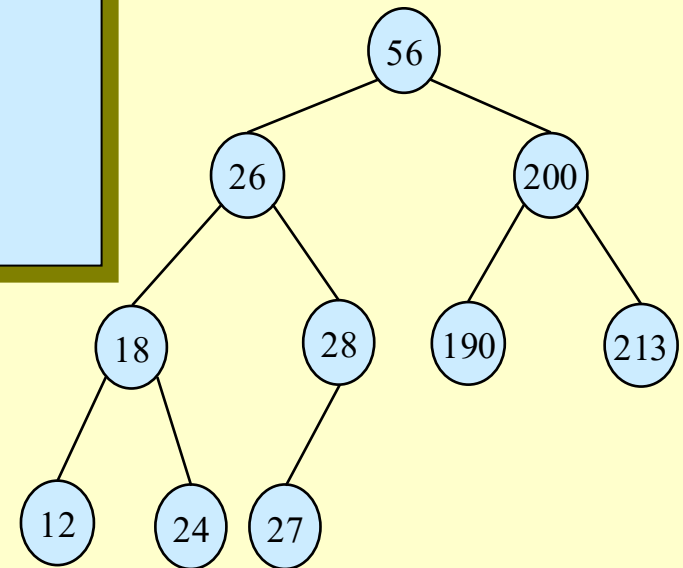
Pseudo-code for Successor

Tree-Successor(x)

- ♦ **if** $right[x] \neq NIL$
- 2. **then** return Tree-Minimum($right[x]$)
- 3. $y \leftarrow p[x]$
- 4. **while** $y \neq NIL$ **and** $x = right[y]$
- 5. **do** $x \leftarrow y$
- 6. $y \leftarrow p[y]$
- 7. **return** y

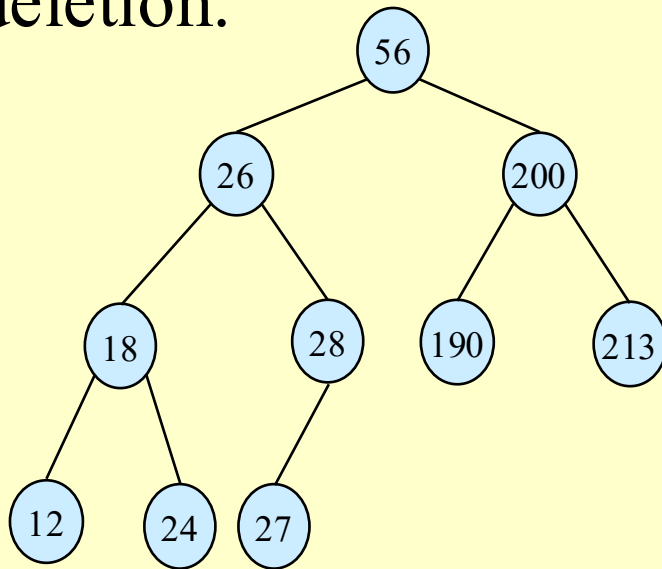
Code for *predecessor* is symmetric.

Running time: $O(h)$



BST Insertion – Pseudocode

- ◆ Change the dynamic set represented by a BST.
- ◆ Ensure the binary-search-tree property holds after change.
- ◆ Insertion is easier than deletion.



Tree-Insert(T, z)

1. $y \leftarrow \text{NIL}$
2. $x \leftarrow \text{root}[T]$
3. **while** $x \neq \text{NIL}$
4. **do** $y \leftarrow x$
5. **if** $\text{key}[z] < \text{key}[x]$
6. **then** $x \leftarrow \text{left}[x]$
7. **else** $x \leftarrow \text{right}[x]$
8. $p[z] \leftarrow y$
9. **if** $y = \text{NIL}$ (inserting first node)
10. **then** $\text{root}[t] \leftarrow z$
11. **else if** $\text{key}[z] < \text{key}[y]$
12. **then** $\text{left}[y] \leftarrow z$
13. **else** $\text{right}[y] \leftarrow z$

Analysis of Insertion

- ♦ Initialization: $O(1)$
 - ♦ While loop in lines 3-7 searches for place to insert z , maintaining parent y .
This takes $O(h)$ time.
 - ♦ Lines 8-13 insert the value: $O(1)$
- ⇒ TOTAL: $O(h)$ time to insert a node.

Tree-Insert(T, z)

```
1.   $y \leftarrow \text{NIL}$ 
2.   $x \leftarrow \text{root}[T]$ 
3.  while  $x \neq \text{NIL}$ 
4.    do  $y \leftarrow x$ 
5.      if  $\text{key}[z] < \text{key}[x]$ 
6.        then  $x \leftarrow \text{left}[x]$ 
7.        else  $x \leftarrow \text{right}[x]$ 
8.   $p[z] \leftarrow y$ 
9.  if  $y = \text{NIL}$ 
10.    then  $\text{root}[t] \leftarrow z$ 
11.    else if  $\text{key}[z] < \text{key}[y]$ 
12.      then  $\text{left}[y] \leftarrow z$ 
13.      else  $\text{right}[y] \leftarrow z$ 
```

Tree-Delete (T, x)

if x has no children ♦ case 0
 then remove x

if x has one child ♦ case 1
 then make $p[x]$ point to child

if x has two children (subtrees) ♦ case 2
 then swap x with its successor
 perform case 0 or case 1 to delete it

⇒ TOTAL: $O(h)$ time to delete a node

Deletion – Pseudocode

Tree-Delete(T, z)

/* Determine which node to splice out: either z or z 's successor. */

- ♦ **if** $left[z] = \text{NIL}$ **or** $right[z] = \text{NIL}$
- ♦ **then** $y \leftarrow z$
- ♦ **else** $y \leftarrow \text{Tree-Successor}[z]$

/* Set x to a non-NIL child of y , or to NIL if y has no children. */

4. **if** $left[y] \neq \text{NIL}$
5. **then** $x \leftarrow left[y]$
6. **else** $x \leftarrow right[y]$

/* y is removed from the tree by manipulating pointers of $p[y]$
and x */

7. **if** $x \neq \text{NIL}$
8. **then** $p[x] \leftarrow p[y]$

/* Continued on next slide */

Deletion – Pseudocode

Tree-Delete(T, z) (Contd. from previous slide)

```
9.   if  $p[y] = \text{NIL}$ 
10.      then  $\text{root}[T] \leftarrow x$ 
11.      else if  $y \leftarrow \text{left}[p[i]]$ 
12.          then  $\text{left}[p[y]] \leftarrow x$ 
13.          else  $\text{right}[p[y]] \leftarrow x$ 
/* If  $z$ 's successor was spliced out, copy its data into  $z$  */
14.  if  $y \neq z$ 
15.      then  $\text{key}[z] \leftarrow \text{key}[y]$ 
16.          copy  $y$ 's satellite data into  $z$ .
17.  return  $y$ 
```

Correctness of Tree-Delete

- ♦ How do we know case 2 should go to case 0 or case 1 instead of back to case 2?
 - » Because when x has 2 children, its successor is the minimum in its right subtree, and that successor has no left child (hence 0 or 1 child).
- ♦ Equivalently, we could swap with predecessor instead of successor. It might be good to alternate to avoid creating lopsided tree.